

Anil Seth



A Quick Start to WebAssembly

Wikipedia defines WebAssembly as a portable stack machine designed to be faster to parse than JavaScript, as well as faster to execute. In this article, the author explores WebAssembly, covering its installation and its relationship with Rust.

Web applications these days are amazing. The first time I became aware of the potential power of Web applications was when I first encountered Gmail. Although Ajax calls had been in use, this was the first application that I knew of that had used them very effectively.

Still, more complex interactions and using local resources needed a plugin. Flash player is both the most used plugin as well as the best example of the problems with plugins. Security issues with Flash never seem to end.

Google tried to overcome some of the issues with the NPAPI plugins with the introduction of NaCl, the native clients. The NaCl clients run in a sandbox, minimising security risks. Google introduced PNaCl, or Portable NaCl, which is an architecture-independent version of NaCl.

Mozilla did not follow Google's lead with a native client, but instead decided to take a different approach, dropping NPAPI from current versions of Firefox.

The solution proposed by Mozilla was *asm.js*, a subset of the JavaScript language, which could run an ahead-of-its-time compiling and optimising engine. A related concept was that you could program in C/C++ and compile the code to *asm.js* using a tool like Emscripten. The advantage is that any application written for *asm.js* would run in any browser supporting JavaScript. However, it would run significantly faster if the browser used optimisation for *asm.js*.

The next step has been the introduction of a byte-code standard for Web browsers, called WebAssembly. The initial implementation targets the *asm.js* feature set and is being developed by all major browsers, including Mozilla, Google, Microsoft and Apple.

As in the case of *asm.js*, you may write the application in C/C++ and use a compiler like Emscripten to create a WebAssembly module.

Installation

The development tools for WebAssembly and Emscripten

are not yet available in the official repositories. You can follow the instructions from <http://webassembly.org/getting-started/developers-guide/> for the installation.

For Linux, you need to build Emscripten from the source. It takes a substantial amount of time for downloading and building it, though.

You can test your installation by trying a simple C program, *hello.c*, as follows:

```
#include <stdio.h>
int main(int argc, char **argv){
    printf("hello, world\n");
}
```

Compile the program as follows to get a JavaScript output:

```
$ emcc hello.c -o hello.js
```

Now, test it and get the expected result.

```
$ node hello.js
hello, world
```

You can check the size of the *hello.js* file and it will be about 320K!

Now, compile the program as follows to get the WebAssembly result:

```
$ emcc hello.c -s WASM=1 -o hello.html
$ ls -lh
-rw-rw-r-- 1 anil anil 85 Aug 24 21:49 hello.c
-rw-rw-r-- 1 anil anil 101K Aug 24 21:50 hello.html
-rw-rw-r-- 1 anil anil 100K Aug 24 21:50 hello.js
-rw-rw-r-- 1 anil anil 45K Aug 24 21:50 hello.wasm
```

You will notice that it creates an *HTML* file, a *js* file and a *wasm* file. The overall size is smaller. You need the *HTML* file, as the *js* file will not execute with the *node*